

Solution Manual: Theory Assignment I

Automata Theory Monsoon 2026, IIIT Hyderabad

Reference: AT-M26-TA1_v2.pdf

Question 1

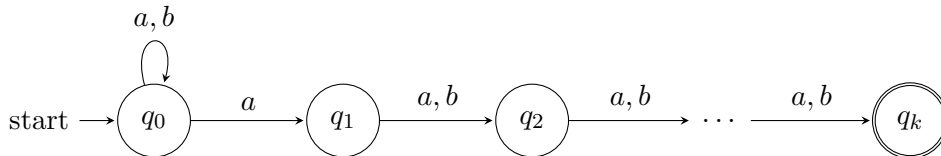
(a) NFA for C_k

The language C_k consists of all strings over $\{a, b\}$ that have an 'a' exactly k places from the right-hand end. We construct an NFA with $k + 1$ states that guesses when it sees this specific 'a' and then transitions through a chain of k states to verify it is followed by exactly $k - 1$ characters (thus making the 'a' the k -th character from the right).

Formal Description: $N = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1, q_2, \dots, q_k\}$
- $\Sigma = \{a, b\}$
- q_0 is the start state.
- $F = \{q_k\}$ is the set of final states.
- $\delta(q_0, a) = \{q_0, q_1\}$, $\delta(q_0, b) = \{q_0\}$
- $\delta(q_i, x) = \{q_{i+1}\}$ for all $x \in \Sigma$ and $1 \leq i < k$
- $\delta(q_k, x) = \emptyset$ for all $x \in \Sigma$

State Diagram:



(b) DFA State Bound for C_k

Proof: We use the Myhill-Nerode theorem (distinguishability argument). Consider the set S of all 2^k possible strings of length exactly k over $\{a, b\}$. We claim that all 2^k strings are pairwise distinguishable with respect to C_k .

Let x and y be two distinct strings in S . Since $x \neq y$, they must differ in at least one position. Let this be the i -th position from the right (where $1 \leq i \leq k$). Without loss of generality, assume x has an 'a' at this position and y has a 'b'.

Let z be any string of length $i - 1$. Append z to both x and y . In the string xz , the character that was i -th from the right in x is now exactly k -th from the right (since $(i) + (i - 1) = k$). Thus, the k -th position from the right in xz is 'a', meaning $xz \in C_k$. In the string yz , the corresponding position has a 'b', meaning $yz \notin C_k$.

Since $xz \in C_k$ and $yz \notin C_k$, z distinguishes x and y . Because all 2^k strings in S are pairwise distinguishable, any DFA recognizing C_k must have at least 2^k states. $\square$

Question 2

Hamming Distance Regularity

Proof: Let L be a regular language recognized by a DFA $M = (Q, \Sigma, \delta, q_0, F)$. We construct an NFA $M' = (Q', \Sigma, \delta', q'_0, F')$ that accepts words at a Hamming distance of at most k from a word in L . M' keeps track of the current state of M and the number of errors (mutations) encountered so far.

- $Q' = Q \times \{0, 1, \dots, k\}$
- $q'_0 = (q_0, 0)$
- $F' = \{(q, e) \mid q \in F \text{ and } 0 \leq e \leq k\}$

The transition function $\delta'((q, e), a)$ is defined for all $a \in \Sigma$ as:

$$\delta'((q, e), a) = \{(\delta(q, a), e)\} \cup \{(\delta(q, b), e + 1) \mid b \in \Sigma, b \neq a \text{ and } e < k\}$$

Correctness: The first component of the transition perfectly simulates the original DFA M when the input matches. The second component non-deterministically "guesses" that a character has been changed from some original character b to the current character a , processes the state change as if b was read, and increments the error counter e . It accepts if the simulated DFA reaches an accepting state and the error counter has not exceeded k . Thus, the language is regular. $\square$

Question 3

DFA Minimization

Based on the provided standard transition diagram, we interpret the states and transitions as follows (with Start=0, Final={2}):

- 0: $0 \rightarrow 1, 1 \rightarrow 5$
- 1: $0 \rightarrow 6, 1 \rightarrow 2$
- 2: $0 \rightarrow 0, 1 \rightarrow 2$
- 3: $0 \rightarrow 2, 1 \rightarrow 6$
- 4: $0 \rightarrow 7, 1 \rightarrow 5$
- 5: $0 \rightarrow 2, 1 \rightarrow 6$
- 6: $0 \rightarrow 6, 1 \rightarrow 4$
- 7: $0 \rightarrow 6, 1 \rightarrow 2$

Step 1: Remove Unreachable States. State 3 has no incoming edges. We remove it. Remaining: $\{0, 1, 2, 4, 5, 6, 7\}$.

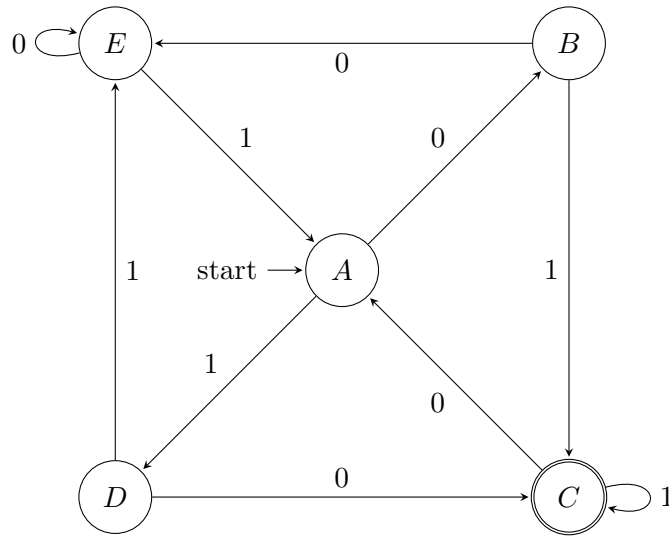
Step 2: Table Filling / Partition Refinement.

- $P_0 = \{2\}, \{0, 1, 4, 5, 6, 7\}$
- Check transitions of $\{0, 1, 4, 5, 6, 7\}$ on 0 and 1:

– $0 \xrightarrow{1} 5$ (non-final); $1 \xrightarrow{1} 2$ (final); $4 \xrightarrow{1} 5$ (non-final); $5 \xrightarrow{0} 2$ (final); $6 \xrightarrow{1} 4$ (non-final); $7 \xrightarrow{1} 2$ (final).

- Split $\{0, 1, 4, 5, 6, 7\}$ into those reaching $\{2\}$ on 1 ($\{1, 7\}$), on 0 ($\{5\}$), and neither ($\{0, 4, 6\}$).
- $P_1 = \{2\}, \{1, 7\}, \{5\}, \{0, 4, 6\}$
- Analyze $\{1, 7\}$: $1 \xrightarrow{0} 6, 1 \xrightarrow{1} 2$ and $7 \xrightarrow{0} 6, 7 \xrightarrow{1} 2$. They behave identically. No split.
- Analyze $\{0, 4, 6\}$: $0 \xrightarrow{0} 1 \in \{1, 7\}$, $0 \xrightarrow{1} 5 \in \{5\}$. $4 \xrightarrow{0} 7 \in \{1, 7\}$, $4 \xrightarrow{1} 5 \in \{5\}$. $6 \xrightarrow{0} 6 \in \{0, 4, 6\}$, $6 \xrightarrow{1} 4 \in \{0, 4, 6\}$.
- Split $\{0, 4, 6\}$ into $\{0, 4\}$ and $\{6\}$.
- $P_2 = \{2\}, \{1, 7\}, \{5\}, \{0, 4\}, \{6\}$. No further splits possible.

Minimized States: $A = \{0, 4\}$ (Start), $B = \{1, 7\}$, $C = \{2\}$ (Final), $D = \{5\}$, $E = \{6\}$.



Question 4

Single-letter Alphabet Regularity $\iff$ Ultimately Periodic

Proof ($\implies$): Suppose $L \subseteq \{a\}^*$ is regular. Let M be a DFA for L with N states. Because the alphabet has only one symbol, the transitions form a sequence of states that must eventually repeat (by the Pigeonhole Principle) forming a single loop. Let the loop start at state $k < N$ and have length $p \geq 1$. For any string of length $n \geq N$, the DFA is inside this loop. Pumping the string by p exactly traverses the loop once more. Thus $a^n \in L \iff a^{n+p} \in L$. Therefore, the set of lengths S is ultimately periodic with threshold N and period p .

Proof ($\impliedby$): Suppose S is ultimately periodic with threshold N and period p . The lengths $< N$ form a finite set $F = \{l_1, l_2, \dots, l_k\}$. The lengths $\geq N$ can be partitioned into a finite set of remainders modulo p , say $R = \{r_1, r_2, \dots, r_m\}$. We can write the language L using a regular expression:

$$L = a^{l_1} \cup a^{l_2} \cup \dots \cup a^{l_k} \cup (a^N (a^p)^* (a^{r_1} \cup \dots \cup a^{r_m}))$$

Since L can be expressed as a regular expression, L is regular. $\square$

Question 5

Language $L = \{a^i b^j \mid i \neq j\}$

(a) Pumping Lemma Proof:

Assume for contradiction that L is regular. Let p be the pumping length. Choose the string $w = a^p b^{p+p!}$. Clearly $w \in L$ since $p \neq p + p!$. By the pumping lemma, $w = xyz$ where $|xy| \leq p$ and $|y| > 0$. Since $|xy| \leq p$, y consists entirely of a 's. Let $y = a^k$ where $1 \leq k \leq p$. We can pump y exactly i times, resulting in $xy^i z = a^{p+(i-1)k} b^{p+p!} \in L$. Choose $i = \frac{p!}{k} + 1$. Because $1 \leq k \leq p$, k perfectly divides $p!$, so i is an integer ≥ 1 . The pumped string has $p + (\frac{p!}{k})k = p + p!$ number of a 's. This yields the string $a^{p+p!} b^{p+p!}$. Here, the number of a 's equals the number of b 's, so it is not in L . This contradicts the pumping lemma. Therefore, L is not regular.

(b) Closure Properties Proof:

Assume L is regular. Regular languages are closed under complementation, so $\bar{L}$ is regular. Regular languages are also closed under intersection. Let us intersect $\bar{L}$ with the regular language $L' = a^* b^*$.

$$\bar{L} \cap a^* b^* = \{a^i b^j \mid i = j\} = L_{eq}$$

We know that L_{eq} is non-regular. Since closure properties resulted in a known non-regular language, our assumption must be false. Thus, L is not regular. $\square$

Question 6

Binary Addition Automaton (B)

We construct a DFA reading columns right-to-left (least significant bit first). The DFA state will keep track of the carry bit.

- $Q = \{C_0, C_1\}$ where C_0 means carry=0, C_1 means carry=1.
- Start state is C_0 . Accepting state is C_0 (so no trailing carry is left over).

For a column input $w = [x, y, z]^T$, the correct addition requires:

$$(x + y + \text{carry}) \bmod 2 = z$$

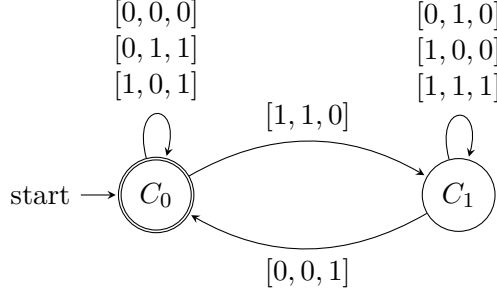
If this holds, the new carry is $\lfloor (x + y + \text{carry})/2 \rfloor$. If it does not hold, the string is rejected (we can implicitly route these to a dead state).

Valid transitions generating no new carry ($C_0 \rightarrow C_0$): $[0, 0, 0], [0, 1, 1], [1, 0, 1]$

Valid transitions generating a new carry ($C_0 \rightarrow C_1$): $[1, 1, 0]$

Valid transitions consuming a carry ($C_1 \rightarrow C_0$): $[0, 0, 1]$

Valid transitions carrying over again ($C_1 \rightarrow C_1$): $[1, 0, 0], [0, 1, 0], [1, 1, 1]$



Question 7

Proving Non-Regularity (Pumping Lemma)

(a) $\{a^{2^n} \mid n \in \mathbb{N}\}$: Assume regular. Let p be the pumping length. Choose $w = a^{2^p}$. Pumping y yields $w_2 = a^{2^p + |y|}$. Since $1 \leq |y| \leq p$, the new length is bounded by $2^p + p$. However, $2^p + p < 2^p + 2^p = 2^{p+1}$. The new length strictly falls between consecutive powers of 2, thus $w_2 \notin L$. Contradiction.

(b) $\{a^m \mid m \text{ is prime}\}$: Assume regular with pumping length p . Choose $w = a^q$ where $q \geq p$ is prime. Pump $y = a^k$ ($1 \leq k \leq p$). Pumping i times yields length $q + (i-1)k$. Choose $i = q+1$. The length is $q + qk = q(1+k)$. Since $k \geq 1$, $1+k \geq 2$, meaning the length is composite. Contradiction.

(c) $\{a^i b^j \mid \gcd(i, j) = 1\}$: Assume regular with pumping length p . Choose $w = a^p b^q$ where $q > p$ is prime. This is in L since $\gcd(p, q) = 1$. Let $y = a^k$ ($1 \leq k \leq p$). Pumping i times yields $a^{p+(i-1)k} b^q$. We want $p + (i-1)k \equiv 0 \pmod{q}$. Since $k < q$ and q is prime, k is invertible modulo q . Choose $i = 1 + (-p \cdot k^{-1} \pmod{q}) + q$. For this positive i , the length of a 's is a multiple of q (say cq). The string is $a^{cq} b^q$, where $\gcd(cq, q) = q > 1$. Contradiction.

(d) $\{\text{bin}(p) \mid p \text{ is a prime number}\}$: Assume regular with pumping length p . Choose a prime $q > 2^p$ whose binary string $w \in L$ has length $> p$. By PL, $w = xyz$. Let y represent a shift of binary data. Pumping y copies a sequence of bits. The numerical value of $xy^i z$ is of the form $V_i = A + (i-1)B$. Specifically, $V_1 = q$. We need $A + (i-1)B \equiv 0 \pmod{q}$. Since $V_1 = q \equiv 0 \pmod{q}$, choosing $i = q+1$ yields $V_{q+1} = q + qB = q(1+B)$. Because $B > 0$, V_{q+1} is a composite integer, meaning its binary representation is not in L . Contradiction.

Question 8

Ambiguous Grammars

A grammar is ambiguous if a string has two different parse trees or leftmost derivations.

(a) $S \rightarrow aAB, A \rightarrow bBb, B \rightarrow A \mid \lambda$

String: $abbbb$.

Derivation 1: $S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abb\lambda bb\lambda = abbbb$.

Derivation 2: $S \Rightarrow aAB \Rightarrow abBbB \Rightarrow ab\lambda bB \Rightarrow abbA \Rightarrow abbbBb \Rightarrow abbb\lambda b = abbbb$.

(b) $S \rightarrow SS \mid \lambda \mid aSb \mid bSa$

String: λ .

Derivation 1: $S \Rightarrow \lambda$.

Derivation 2: $S \Rightarrow SS \Rightarrow S\lambda \Rightarrow \lambda\lambda = \lambda$.

(c) $S \rightarrow aSb \mid SS \mid \lambda$

String: ab .

Derivation 1: $S \Rightarrow aSb \Rightarrow a\lambda b = ab$.

Derivation 2: $S \Rightarrow SS \Rightarrow aSbS \Rightarrow a\lambda bS \Rightarrow ab\lambda = ab$.

Question 9

Pushdown Automata (PDA)

(a) $L = \{a^m b^n \mid m \leq n \leq 2m\}$

Logic: For every a , non-deterministically push either one or two stack symbols X . For every b , pop exactly one X . Accept by empty stack.

- $q_0 \xrightarrow{\epsilon, \epsilon \rightarrow \$} q_1$ (Init bottom marker)
- $q_1 \xrightarrow{a, \epsilon \rightarrow X} q_1$ (Push 1)
- $q_1 \xrightarrow{a, \epsilon \rightarrow XX} q_1$ (Push 2)
- $q_1 \xrightarrow{\epsilon, \epsilon \rightarrow \epsilon} q_2$ (Transition to b's)
- $q_2 \xrightarrow{b, X \rightarrow \epsilon} q_2$ (Pop matching b's)
- $q_2 \xrightarrow{\epsilon, \$ \rightarrow \epsilon} q_f$ (Accept by empty stack / final state)

(b) $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$

Logic: Start state splits non-deterministically via ϵ into two independent PDAs.

- **Branch 1** ($i = j$): Push X for a , pop X for b , ignore c (read without interacting with stack). Accept if stack empties when b 's finish.
- **Branch 2** ($j = k$): Ignore a (read without pushing). Push X for b , pop X for c . Accept if stack empties.

(c) Conversion (Final State to Empty Stack)

If a PDA P accepts by final states F , convert it to accept by empty stack: 1. Add a new start state S_{new} that pushes a special marker Z_0 onto the empty stack, then transitions via ϵ to the old start state S_{old} (not necessary if they ϵ -transition from the old final state to the new final state). 2. Add a new single accepting state Q_{empty} . 3. From every final state $q_f \in F$, add an ϵ -transition (without popping) to Q_{empty} . 4. At Q_{empty} , add loops that pop every possible stack symbol $X \in \Gamma$, plus Z_0 . Once Z_0 is popped, the stack is completely empty, strictly forcing empty-stack acceptance.

Question 10

Middle Third Removal Operation ($A_{1/3-1/3}$)

Proof: We will define a specific regular language A and show that removing its middle third results in a non-regular language.

Let $A = 0^*10^*20^*$. Clearly, A is regular. Every string in A has exactly one 1 and exactly one 2. A string $w \in A$ has length $|w| = 3n$. We remove the middle third y (where $|x| = |y| = |z| = n$) to form the string $xz \in A_{1/3-1/3}$.

Intersect $A_{1/3-1/3}$ with the regular language $R = 0^*120^*$. If $xz \in R$, then x must end with 1 and z must start with 2. Because $|x| = |z|$, x and z must have the exact same number of leading/trailing 0's. Thus, $x = 0^k1$ and $z = 20^k$ for some $k \geq 0$.

The length of xz is $2k + 2$. Thus $n = k + 1$. The removed middle third y must have length $k + 1$. The original string was $xyz = 0^k1y20^k$. For this string to belong to $A = 0^*10^*20^*$, the sub-string y can only consist of zeroes. So $y = 0^{k+1}$. This is a valid string in A .

Therefore, the intersection is exactly:

$$A_{1/3-1/3} \cap R = \{0^k120^k \mid k \geq 0\}$$

This resulting language is a standard known non-regular language (provable trivially via the pumping lemma, similar to 0^n1^n). Because regular languages are closed under intersection, and R is regular, $A_{1/3-1/3}$ must be non-regular to produce a non-regular intersection. $\square$